



指令系统

2025年秋

主要内容

□ 指令系统概念

- 基本概念
- 指令系统设计要求
- 指令功能和分类

□ 指令格式

- 变长指令字/定长指令字
- 操作码扩展

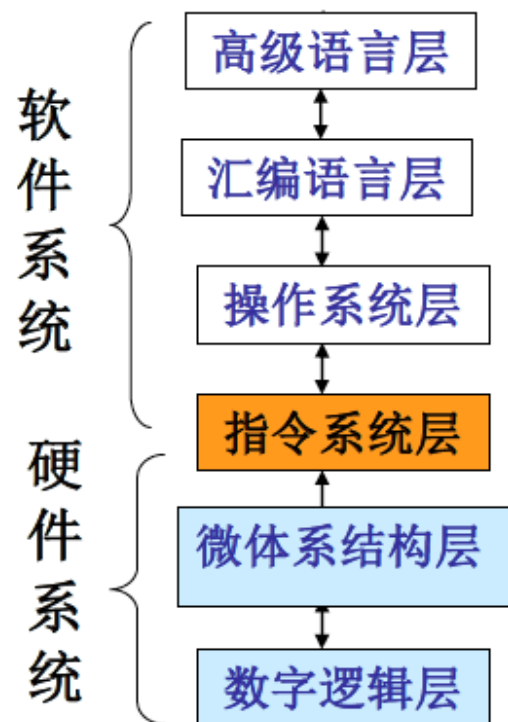
□ 寻址方式

指令、指令集与指令系统

- ❑ 计算机系统由硬件和软件两大部分组成。硬件指由中央处理器、存储器以及外围设备等组成的实际装置。软件是为了使用计算机而编写的各种系统的和用户的程序，程序由一个序列的计算机指令组成。
- ❑ 指令是计算机运行的最小的功能单元，是指挥计算机硬件运行的命令，是由多个二进制位组成的位串，是计算机硬件可以直接识别和执行的一个信息体。
- ❑ 一台计算机提供的全部指令构成该计算机的指令集（IS）。指令用于程序设计人员告知计算机执行一个最基本运算、处理功能，多条指令可以组成一个程序，完成一项预期的任务。
- ❑ 指令系统（ISA）给出指令集中每条指令的规范，还需要明确系统特权级、异常/中断、存储管理等实现要求。
- ❑ ABI（Application Binary Interface）是机器语言级别上不同程序模块之间进行调用时共同遵循的一些约定。

指令系统的地位

- ❑ 指令系统层处在硬件系统和软件系统之间，是硬、软件之间的接口部分，对两部分都有重要影响。
- ❑ 硬件系统用于实现每条指令的功能，解决指令之间的衔接关系；
- ❑ 软件由按一定规则组织起来的许多条指令组成，完成一定的数据运算或者事务处理功能。
- ❑ 指令系统优劣是一个计算机系统是否成功的关键因素。



计算机系统的层次结构

指令系统设计要求

□ 完备性

- 指令功能齐全、编程方便
- 提供对操作系统实现的必要支持（中断/异常、MMU、特权级）

□ 规整性

- 指令格式简单、统一

□ 高效性

- 占内存少，运行高效

□ 兼容性

- 同一系列软件兼容
- 跨指令集的翻译

□ 扩展性

- 模块化设计

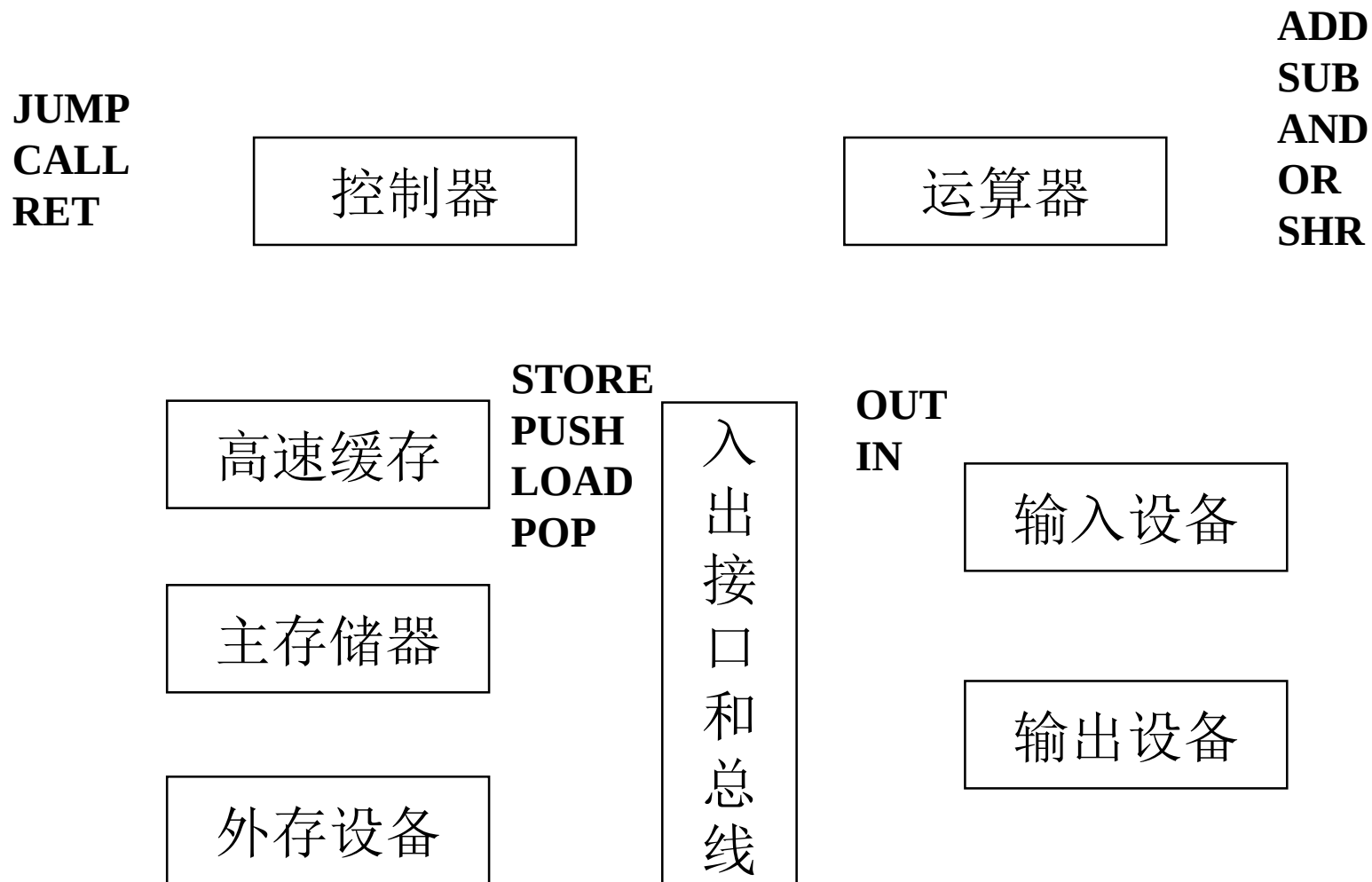
□ 适应性

- 对过时技术的扬弃

计算机中需配备的指令

- ❑ 指令是用户使用计算机和计算机本身运行的最小的功能单元：①指令是由多个二进制位组成的数串，②用于设计程序，③计算机硬件可直接识别和执行。通常情况下一台计算机需要提供哪些指令呢？
- ❑ 计算机用于计算和处理数据，为此，要在计算机硬件系统中设置5种类型的部件：运算器部件、控制器部件、存储器部件、输入设备、输出设备，各自承担数据运算、系统指挥控制、保存当前程序和数据、执行输入和执行输出的功能。需要在计算机中设置为使用和控制这几个部件运行的相应指令。
 - 支持OS实现的指令

使用硬件系统的基本指令



指令的功能和分类

□ 指令用于设计程序，指令系统构成最低级别的程序设计语言，程序设计人员通过指令直接指挥计算机的硬件完成某一个基本的运算、处理功能，例如：

- 对数值数据的算术运算，对逻辑数据的逻辑运算，
- 在计算机部件之间传送、保存数据，
- 从外部向计算机内输入数据，
- 把计算机内部计算结果输出出来，
- 按照某种条件控制计算机选择执行某段程序，
- 当然还有另外一些方面的更深层次的要求等；
- 可以按照指令执行的功能对它们进行分类。

指令的功能和分类

□ 算术与逻辑运算指令

- 加、减、乘、除、变符号等算术运算
- 与、或、非、异或等逻辑运算

□ 移位操作指令

- 算术移位（一般只右移）、逻辑移位、循环移位

□ 数据传送指令

- 通用寄存器之间传送
- 通用寄存器与主存储器存储单元之间传送
- 主存储器不同存储单元之间传送

□ 输入输出指令

- 通用寄存器与输入输出设备（接口）之间传送

指令的功能和分类

□ 转移指令

- 变动程序中指令执行次序的指令，分为无条件转移指令和条件转移指令

□ 子程序调用与返回指令

- 调用指令与返回指令二者要配合使用，子程序的最后一条指令一定是返回指令，执行结束后返回主程序断点

□ 堆栈操作指令

- 堆栈（**stack**）是由若干个连续存储单元组成的先进后出的存储区，有压入（即进栈）和弹出（即退栈）操作

□ 其他指令

- 置条件码指令、开中断指令、关中断指令
- 停机指令、空操作指令、特权指令

指令表示

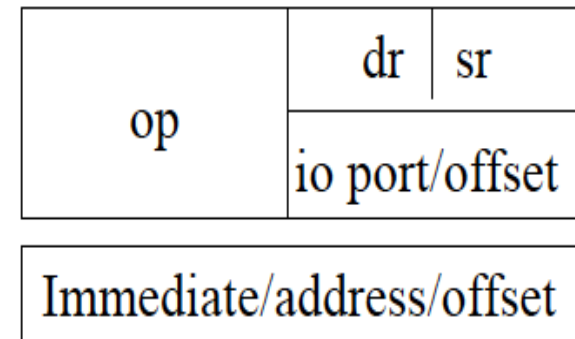
- 指令中的内容，包括指令操作码（指出指令完成的运算处理功能和数据类型）和操作数或指令的地址（指明用到的数据或地址）两部分。例如：
 - 算逻运算中的运算功能，数据来源或结果去向
 - 数据传送指令中的数据原来位置和新的存储位置
 - 输入输出指令中用到的设备和数据来、去的位置
 - 转移指令的转移类别、转移条件和转移地址等
- 每一条指令必须指明它需要完成的功能，通常用几位指令操作码表示；还需要指明用到的数据、地址或设备，通常在地址字段给出，可能是：
 - (1)寄存器编号，(2)设备端口地址，
 - (3)存储器的单元地址(4) 数值等几种信息。

指令格式与指令字长

□ 指令字长是指组成一条指令的二进制数的位数，例如8 bits、16 bits、32 bits、64 bits等，指令格式与指令字长密切相关，指令字越长可以给出的信息越多。一个指令字通常由指令操作码和操作数地址两部分组成，如何把一个指令字划分成多个字段并分配各字段所表示的内容大有学问。

R 型	funct7	rs2	rs1	funct3	rd	opcode
I 型	imm[11:0]		rs1	funct3	rd	opcode
S 型	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
SB / B 型	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode
UI / J 型	imm[20,10:1,11,19:12]				rd	opcode
U 型	imm[31:12]				rd	opcode

RISC-V 指令格式 (32位)



TEC-2000 指令格式 (16位)

指令格式

- 指令字：完整的一条指令的二进制表示
- 指令字长：指令字中二进制代码的位数
 - 机器字长：计算机能直接处理的二进制数据的位数
 - 指令字长（字节倍数）= **0.5、1、2...**个机器字长
 - 定长指令字结构**vs.** 变长指令字结构
- 指令格式：指令字中操作码和操作数地址的二进制位的分配方案



- 操作码：指明本条指令的操作功能，
- 每条指令有一个确定的操作码
- 操作数地址：说明操作数存放的地址，有时是操作数本身

操作码组织与编码

□ 定长的操作码的组织方案

- 在指令字固定位置分配若干位用于表示操作码，有利于简化计算机硬件设计，提高指令译码和识别速度
- 例如：**IBM360**机、**THINPAD**教学机

□ 变长的操作码的组织方案

- 在指令字最高位部分用一固定长度的字段来表示基本操作码，而对于部分操作数地址位数可以少的指令，则把另外多位辅助操作码扩充到该操作数地址字段，即操作码位数可变。
- 这种方法在不增加指令字长的情况下，可表示更多的指令，但增加了译码和分析难度，要求更多的硬件支持
- 例如：**PDP-11**计算机、**TEC-2000**的8位机

操作码组织与编码

- ❑ 操作码字段与操作数地址字段有所交叉的方案
- ❑ 不同指令的操作码长度可以不同，表示操作码所用的一些二进制位不再集中在指令字的最高位部分，而是与用于表示操作数地址的一些字段有所交叉，操作码还被区分为主操作码和辅助操作码这样不同的两部分，这是一种比较特殊、不很常用的方案。
- ❑ 例如：**NOVA (DJS-130)** 计算机就采用这种方案

指令操作码的位数限制指令系统中的指令条数！

操作数个数与来源

□ 指令操作数个数

- 无操作数指令（零地址指令）
- 单操作数指令（一地址指令）
- 双操作数指令（二地址指令）
- 三操作数指令（三地址指令）
- 多操作数指令（多地址指令）

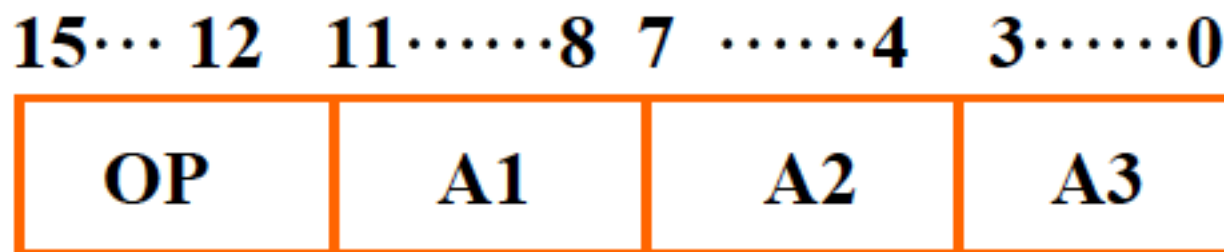
□ 指令操作数来源和去向

- CPU内部的通用寄存器
- 输入输出设备（接口）的一个寄存器
- 主存储器的一个存储单元

OP			
OP	A ₁		
OP	A ₁	A ₂	
OP	A ₁	A ₂	A ₃
OP	A ₁	A ₂	更多

指令操作码的扩展技术

- 假设某机器的指令长度为**16**位，包括**4**位基本操作码和三个**4**位地址码段。



- **4** 位基本操作码可表示**16**个状态，
- 如用**4** 位操作码，则能表示**16** 条三地址指令，
- 若用**8** 位操作码，则可表示**256** 条二地址指令，
- 而用**12**位操作码，则可表示**4096**条一地址指令，
- 若**16**位全用作操作码，则可表示**65536**条零地址指令

指令操作码的扩展技术

- ❑ 若需要在**16**位字长的指令中能够同时支持三地址、二地址、一地址指令各**15**条，零地址指令**16**条，则可以选择如下方案的变长操作码实现：
- ❑ 15条三地址指令的操作码为：0000 ~ 1110
- ❑ 15条二地址指令的操作码的高4位选用1111，低4位用0000 ~ 1110，即得到：11110000 ~ 11111110
- ❑ 15条一地址指令的操作码的高8位选用11111111，低4位用0000 ~ 1110，即：111111110000 ~ 111111111110
- ❑ 16条零地址指令的操作码的高12位每位均用1，低4位随意，即：1111111111110000 ~ 1111111111111111

指令操作码扩展技术

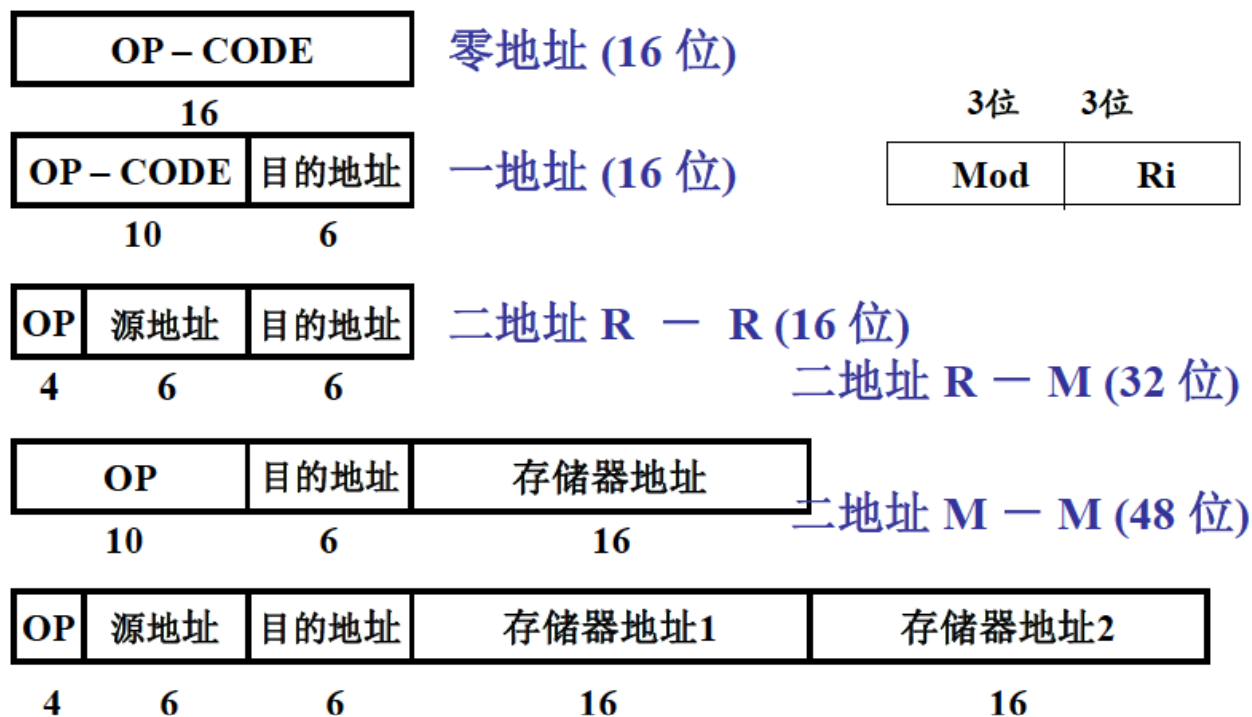
□ 前面介绍的操作码扩展方案中，每次扩展4位并仅保留了一个编码用于接下来的扩展过程，当每次扩展的位数和保留的位数变化时，后面可扩展的指令条数就可以变化。例如在16位字中的指令字中，可以选用如下方案支持三地址指令、二地址指令、一地址指令和零地址指令14、30、31、16条：

- 14条三地址为：0000 ~ 1101 (保留1110、1111 两个码)
- 30条二地址为：11100000 ~ 11111101 （保留2个码）
- 31条一地址为：111111100000 ~ 111111111110 (保留1个码)
- 16条零地址为：1111111111110000 ~ 1111111111111111

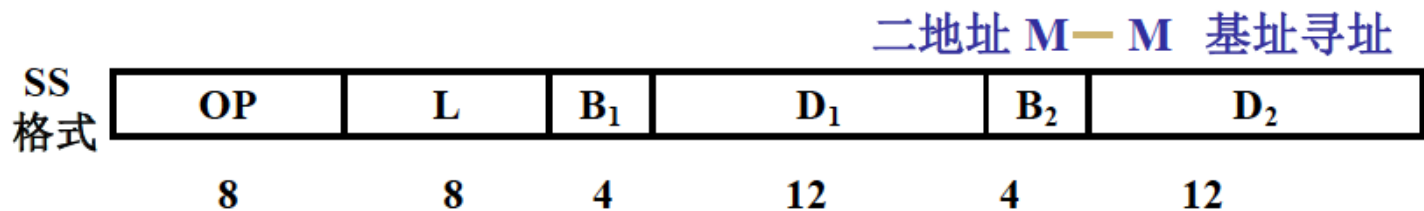
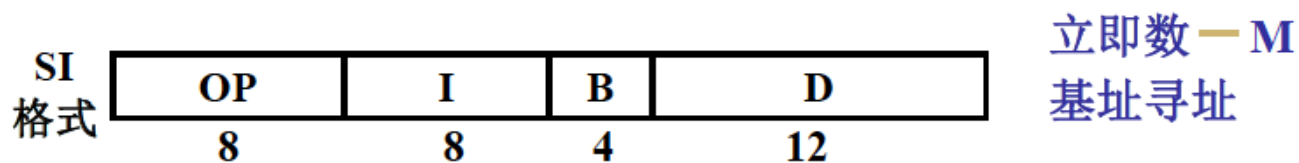
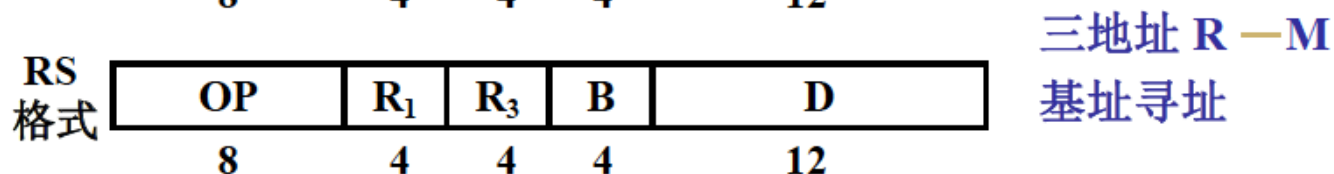
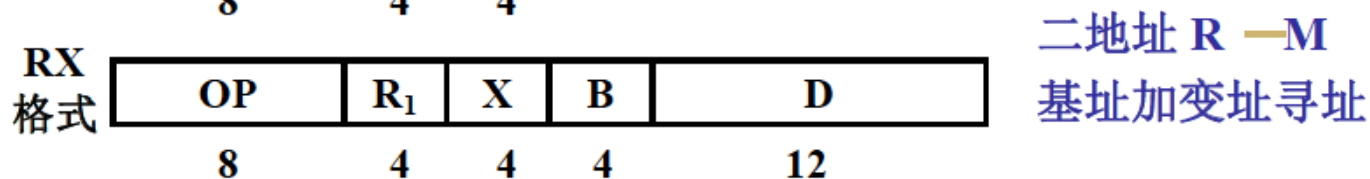
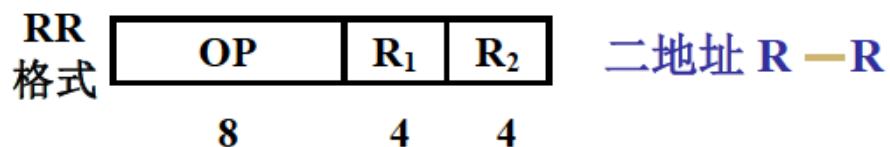
指令操作码扩展技术

□ (PDP-11 指令为例)

□ 指令字长有16 位、32 位、48 位三种(1字、2字、3字)



IBM 360指令格式



寻址方式

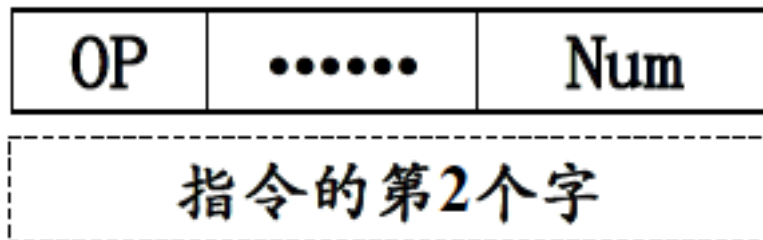
- 寻址方式（又称编址方式）指的是确定本条指令的操作数地址及下一条要执行的指令地址的方法。
- 不同的计算机系统,使用数目和功能不同的寻址方式，其实现的复杂程度和运行性能各不相同。有的计算机寻址方式较少，而有些计算机采用多种寻址方式。
- 通常需要在指令中为每一个操作数专设一个地址字段，用来表示数据的来源或去向的地址。在指令中给出的操作数（或指令）的地址被称为形式地址，使用形式地址信息并按一定规则计算出来或读操作得到的一个数值才是数据（或指令）的实际地址。在指令的操作数地址字段，可能要指出：
 - ①运算器中的累加器的编号或专用寄存器名称（编号）
 - ②输入/输出指令中用到的I/O 设备的入出端口地址
 - ③内存储器中的一个存储单元（或一I/O设备）的地址
- 有多种基本寻址方式和某些复合寻址方式。

立即数寻址

□ 所需的一个操作数在指令的地址字段部分直接给出。

□ Num 即为操作数的值

□



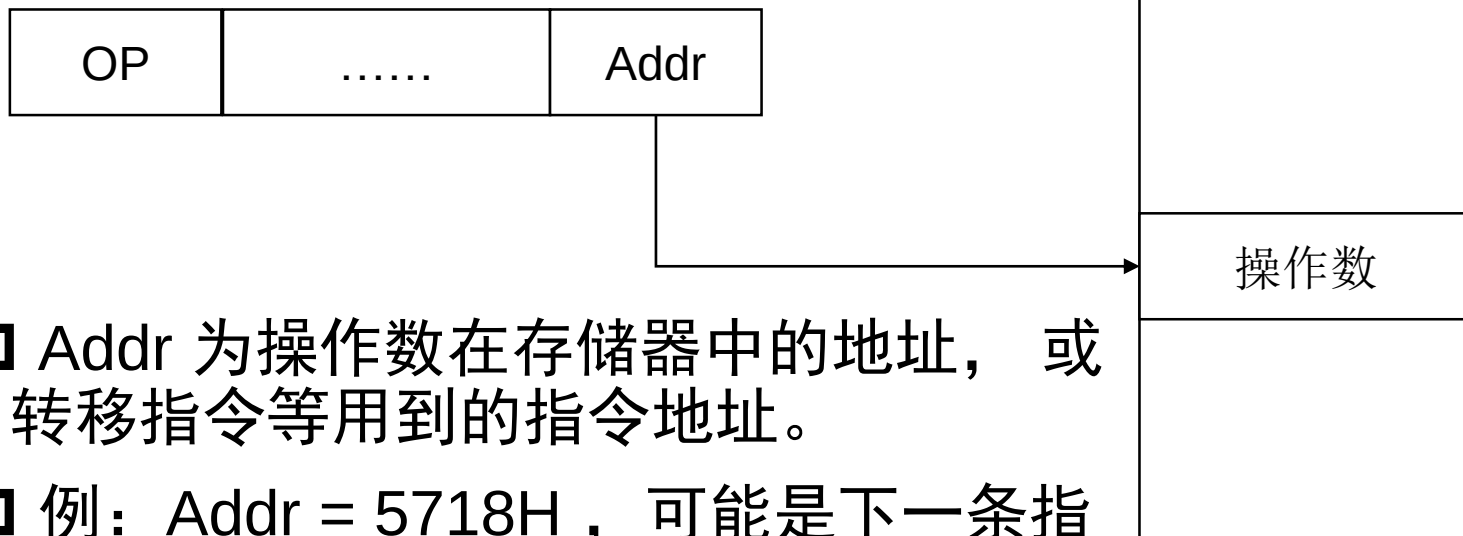
- 适用于操作数固定的情况，取指同时取得操作数，指令执行阶段不必到存储器中取操作数，提高了指令执行速度；
- 当该立即数的值较小(占用位数少)时，可在指令字第一个字中直接给出，否则需要用指令的第二个字提供。

□ 例：Num = 1234H，指令的一个操作数就是1234H

□ 这里的H 表示1234 是16 进制的值

直接寻址

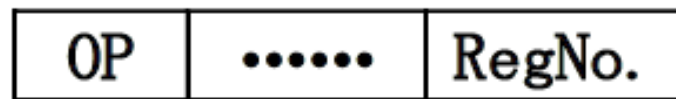
- 在指令的地址码字段，直接给出所需的操作数(或指令)在存储器中的地址。



- Addr 为操作数在存储器中的地址，或转移指令等用到的指令地址。
- 例：Addr = 5718H，可能是下一条指令的地址或一个操作数的地址，若 [5718H] = 3，则用5718H 作地址，从内存储器单元中读出的操作数就是3。

寄存器寻址/寄存器间址

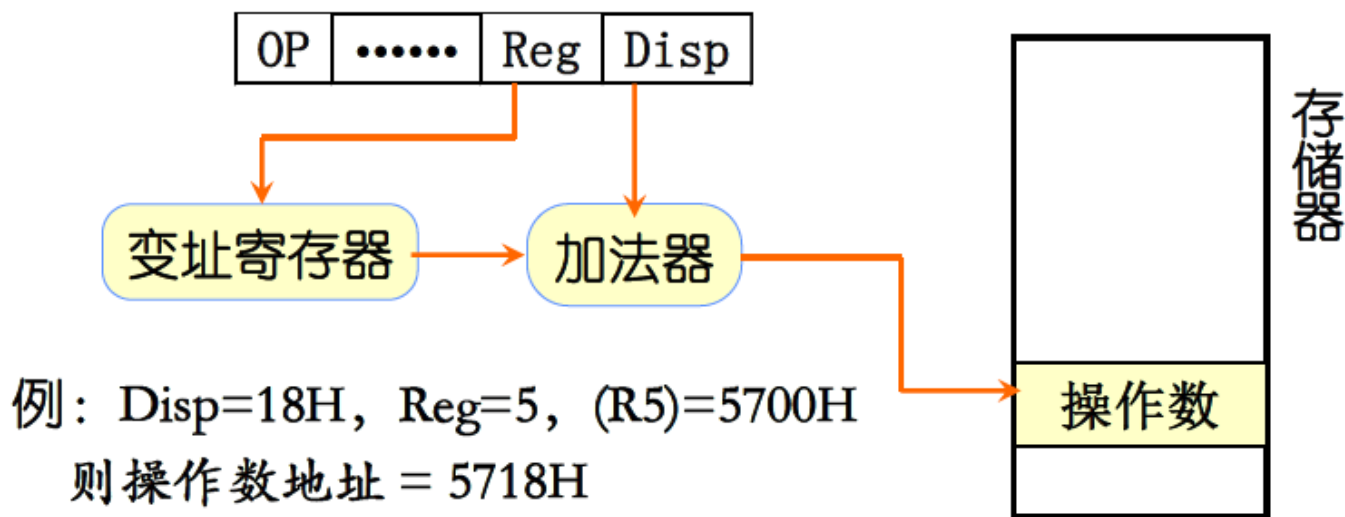
- 计算机的CPU中设置有一定数量的通用寄存器，用于存放操作数、操作数地址或中间结果。假如指令地址码字段给出某一通用寄存器的编号(地址)，且所需的操作数就在这一寄存器中，这就是寄存器寻址方式；若该寄存器中存放的是操作数在内存存储器中所在单元的地址，这就是寄存器间接寻址方式。可通过指令的操作码或另设一个字段，来区分这两种不同的寻址方式。



- 例：RegNo.=5，使用5# 累加器，
- 此时若5# 累加器中的内容为7，可记为(R5)=7，
- 对寄存器寻址，操作数就是寄存器中的数值7
- 对寄存器间接寻址，从内存7# 单元读出来的数才是操作数

变址寻址

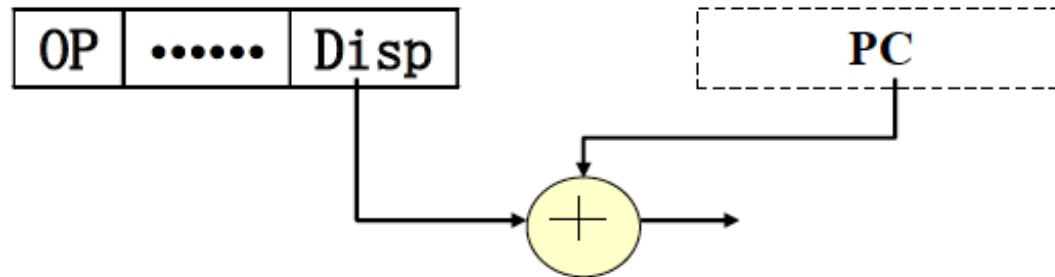
- 操作数的地址由指定的变址寄存器（由Reg指定）的内容和指令中的变址偏移量（Disp）相加得到。



- 变址寄存器内容变化，变址偏移量不变，便于读写数组中的元素，是计算机中常用的一种寻址方式。

相对寻址

- 指令的地址由程序计数器PC 的内容（即当前执行指令的地址）和指令的相对寻址偏移量相加得到。

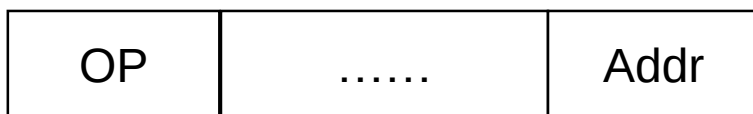


例：Disp = 48H (PC) = 5600H
则实际地址 = 5648H

- （1）主要用于转移指令，对浮动程序很有用。
- （2）偏移量可正可负，通常用补码表示。

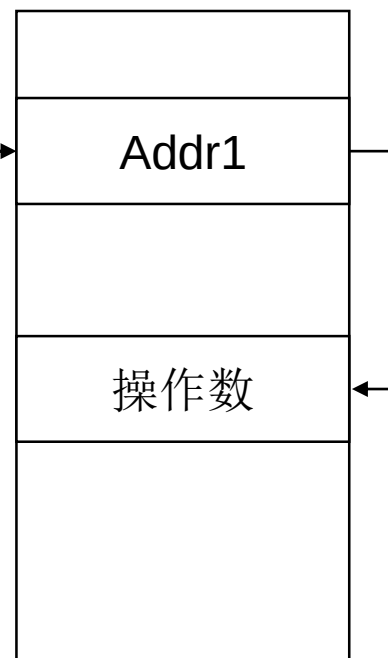
间接寻址

- 指令的地址码字段给出的内容既不是操作数，也不是操作数的地址，而是操作数（或指令）地址的地址，这被称为间接寻址方式，多一次读内存存储器的操作。



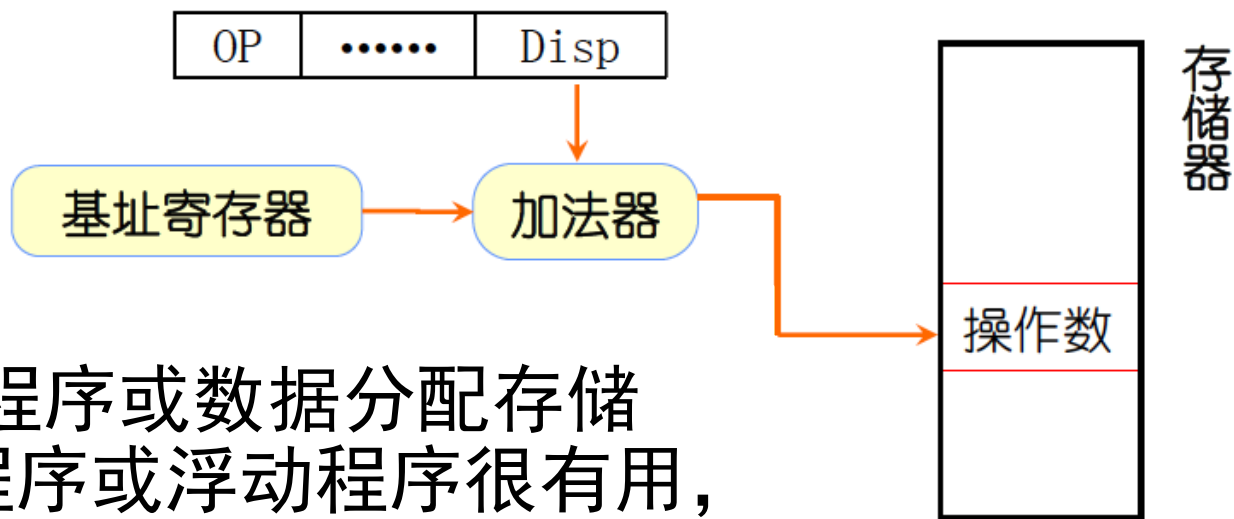
- 指令中的Addr 可以用其他寻址方式给出，例如变址寻址，这就成为变址寻址与间接寻址的复合寻址方式

存储器



基址寻址

- 在计算机中设置一个专用的基址寄存器，操作数（或指令）的地址通过基址寄存器的内容和指令中的地址码相加得到。

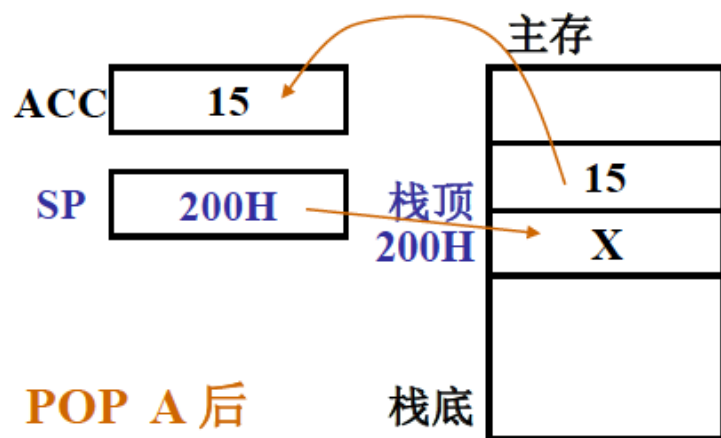
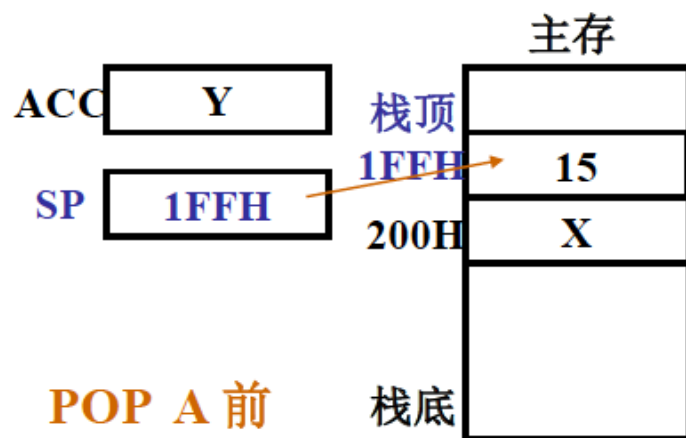
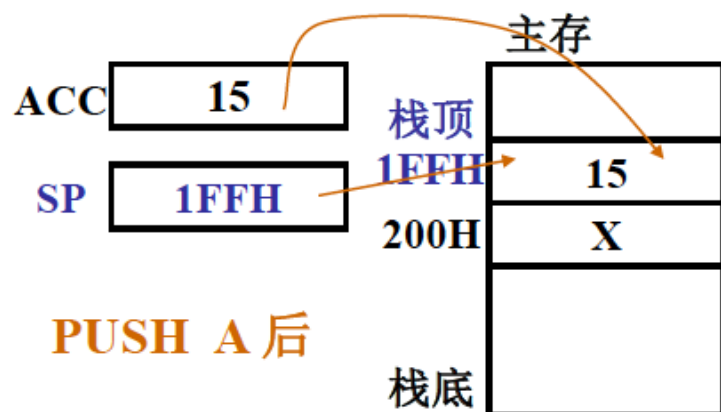
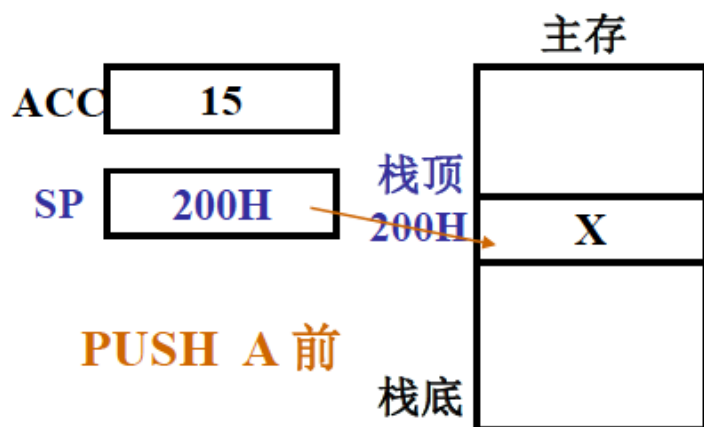


- 主要用于为程序或数据分配存储区，对多道程序或浮动程序很有用，解决了程序在存储器中的定位和扩大寻址空间等问题。

堆栈寻址

- ❑ 堆栈是内存存储器中一块按“后进先出”原则进行读写的存储区，并通过一个专用的寄存器(称为**堆栈指针SP**)给出堆栈的栈顶地址，执行读写堆栈操作通常总在栈顶进行，故不必在指令中给出堆栈地址，而且在读写操作的前后伴随有自动修改SP内容的动作，确保使SP总是指向堆栈的栈顶。例如，**按字寻址**时：
 - ❑ **入栈操作**: $SP - 1 \rightarrow SP$ 和 AR ，即SP的内容减1存回SP,并送入内存地址寄存器，接下来才可以把数据写到堆栈中，这是因为需要把数据写到新开辟出来的栈顶单元中。
 - ❑ **出栈操作**: $SP \rightarrow AR$ ，完成一次读堆栈操作后，还要执行一次 $SP + 1 \rightarrow SP$ 的操作，用于修改SP内容，这是因为数据读出后原来它的下一个相邻单元变成为栈顶。

堆栈寻址举例



异常

□ 概念

- 程序运行时发生各种意外，造成程序不能正常运行
- 由操作系统进行处理
- 返回程序继续执行

□ 异常分类

- 中断：外部事务
- 异常：CPU内部原因
- 陷进：程序通过指令触发

□ 异常响应

- 软硬件协同响应
- 由指令系统进行规范

存储管理

□ 目标

- 实现多道程序间的内存保护、共享
- 确保系统安全

□ 方法

- 操作系统，硬件提供支持
- 段式管理、页式管理
- 段表、页表、TLB

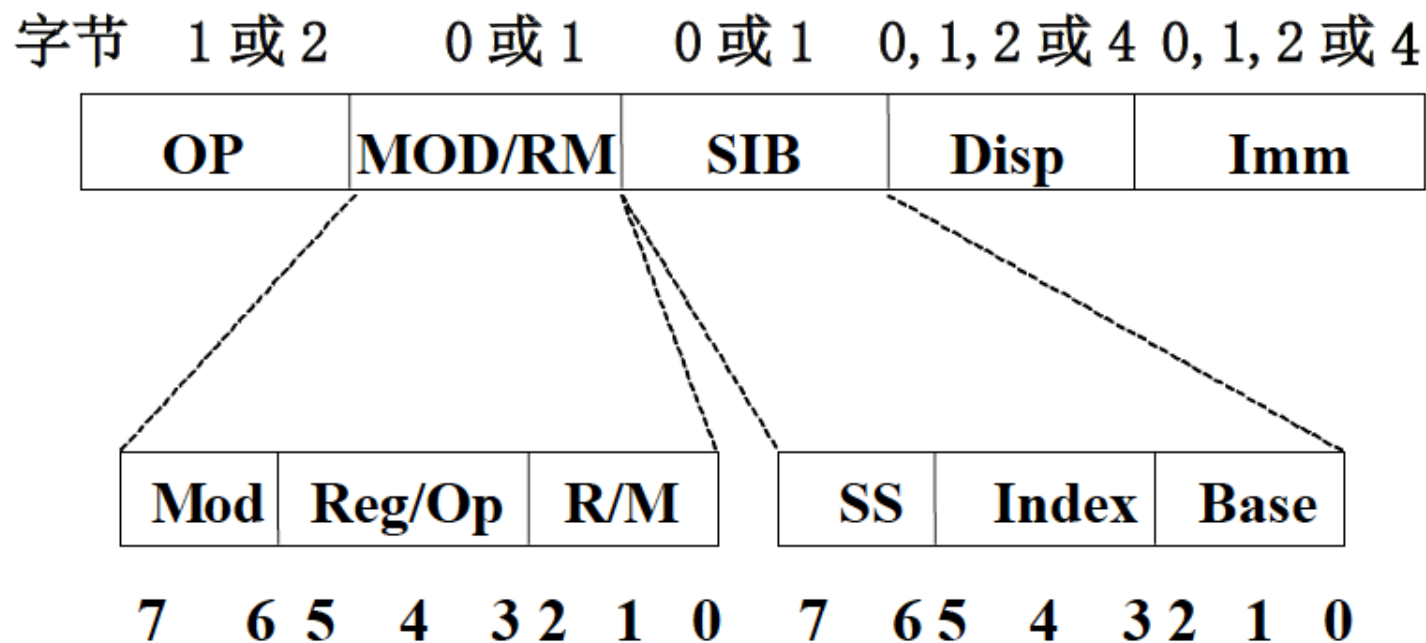
特权级

- ❑ 操作系统进行系统管理
- ❑ 应用程序使用系统资源
- ❑ 权限级别设定
- ❑ 软/硬件协同实现

应用程序二进制接口（ABI）

- ABI是指在特定的ISA和特定的操作系统下，各类程序运行时，尤其是相互调用时所遵循的机器语言层级的接口约定。
 - 寄存器使用约定
 - 函数调用参数和返回值传递约定
 - 系统调用参数和调用号约定
 - 目标文件二进制格式
 - 程序的虚拟地址空间排布
 - 不同数据类型数据占用内存空间的大小。
- ABI规范了应用程序调用底层操作系统的接口标准，是计算机系统程序员必须了解和遵循的规则。

x86 指令系统



x86指令的格式

- ❑ **OP**: 指令操作码, 某些指令在操作码中还包含有操作数长度或立即数是否需扩充符号位(S) 等信息;
- ❑ **MOD / RM**: 与下一字节提供寻址信息。**MOD / RM** 指出操作数在寄存器中还是在存储器中。该字节分成3个字段: **Mod**字段与**R / M**字段可以产生32个编码, 分别表示8个寄存器和24种变址方法; **Reg / op**字段可以是寄存器号, 或者作为3位附加的操作码; **R / M**字段是1个操作数所在寄存器或者与Mod字段一起指出寻址方式。
- ❑ **SIB**: 当MOD / RM为某些值时, 需SIB 参与决定寻址方式。SIB分成3个字段, **SS** 指出变址寄存器的放大因子; **Index** 指出变址寄存器; **Base** 指出基址寄存器。
- ❑ **Disp**: 当寻址方式指示用到disp(位移量)时, 存在8位, 16位或32位的位移量3种情况。
- ❑ **Imm**: 对寻址方式是立即数, 分为8位, 16位或32位3 种。
- ❑ **前缀**: 指令前缀、段前缀、操作数前缀、地址长度前缀

x86指令寻址方式

□ 1. 立即数

□ 2. 寄存器

□ 3. 直接

$$E = \text{Disp}$$

□ 4. 基址

$$E = (B) \quad B \text{基址寄存器}$$

□ 5. 基址 + 偏移量

$$E = (B) + \text{Disp}$$

□ 6. 比例变址 + 偏移量

$$E = (I) * S + \text{Disp}$$

□ I 变址寄存器, S为比例因子 (1,2,4,8)

□ 7. 基址 + 变址 + 偏移

$$E = (B) + (I) + \text{Disp}$$

□ 8. 基址 + 比例变址 + 偏移

$$E = (B) + (I)*S + \text{Disp}$$

□ 9. 相对

$$\text{指令地址} = (PC) + \text{Disp}$$

小结

□ 指令系统

- 是硬件和软件的接口
- CISC和RISC

□ 指令格式

- 指令如何用二进制编码表示，主要是操作码和操作数地址的安排方案

□ 寻址方式

- 操作数寻址方式
- 对操作系统、编译程序提供支持，方便程序员使用计算机
- 寻址方式的选择

阅读和思考

□ 阅读

□ 思考

- 计算机指令系统中哪些是必备指令?为什么?
- 指令寻址方式有哪些?这些寻址方式可以在高级语言程序中找到哪些影子?
- 根据ThinPAD RISC-V指令系统要求, 思考ALU应具备的功能。

谢谢

x86指令系统特点

- ❑ 指令格式复杂
- ❑ 寻址方式多样
- ❑ 通用寄存器较少
- ❑ 兼容直到8086
- ❑ 编译系统复杂
- ❑ 指令流水实现复杂
- ❑ 典型CISC 指令集

现代x86指令系统

□ 系统的运行模式

- 实模式，兼容16位的物理地址模式
- 32位保护模式，分页分段模式（地址扩展模式）
- 64位平坦模式，分段模式禁止
- SMM模式（用以处理紧急情况）
- 虚拟机模式
- SGX兼容运行环境

□ 系统的指令

- 应用程序指令，SIMD指令
- 模式相关指令
- 内存相关指令
- IO相关指令

系统的运行模式

- 决定当前处理器的运行的系统上下文，对相同的指令可能有不同的解释，使用不同的物理资源
 - 运行在实模式：直接访问物理地址，只能用16位地址模式
 - 运行在保护模式：必须通过段页式转换，才能访问物理地址
 - 运行在虚拟机模式：经过两层的地址转换，才能访问物理地址

不同的x86指令类别

- ❑ General Purpose Instructions: 算术指令，IO指令，跳转分支指令
- ❑ 系统级指令
- ❑ x87 FPU指令
- ❑ SIMD: MMX, SSE, AVX
- ❑ 加解密指令，密码学哈希指令
- ❑ TSX硬件事务处理指令
- ❑ VMX 虚拟机扩展指令
- ❑ SMX安全执行扩展指令
- ❑ SGX用户级安全执行环境指令

x86指令举例

- ❑ 短的指令：int 3，指令码为0xcc
- ❑ The maximum length of an Intel 64 and IA-32 instruction remains 15 bytes.(from the manual)
- ❑ AVX指令可以到这个长度，超过这个长度会抛出非法指令异常
- ❑ 按照不同的方法，统计结果不一样，粗的有1000条左右，特别细致的话会有3000条左右指令
- ❑ 最新的手册有4922页（指令手册加在一起）
- ❑ 除了指令本身外，指令所处的运行模式，寻址方式也造成了指令执行的复杂度
- ❑ 作为对比：MIPS32指令有150条左右

数据移动指令

指令	说明
MOV DST, SRC	数据从SRC复制到DST
PUSH SRC	把SRC 压入堆栈
POP DST	从堆栈弹出一字存入DST
XCHG DS1, DS2	交换 DS1 和 DS2
LEA DST, SRC	把SRC的有效地址存储DST
CMOV DST, SRC	条件复制

算术指令

指令	说明
ADD DST, SRC	把SRC加到DST中
SUB DST, SRC	从SRC减去DST
MUL SRC	EAX乘以SRC（无符号）
IMUL SRC	EAX乘以SRC（带符号）
DIV SRC	EDX : EAX 除以 SRC（无符号）
IDIV SRC	EDA : EAX 除以 SRC（带符号）
ADC DST, SRC	把SRC加到DST中，并加进位位
SBB DST, SRC	从SRC中减去DST和进位位
INC DST	DST 加 1
DEC DST	DST 减 1
NEG DST	DST取反（也就是0-DST）

逻辑指令

指令	说明
AND DST, SRC	SRC 和 DST进行逻辑与，结果存入 DST
OR DST, SRC	SRC 和 DST进行逻辑或，结果存入 DST
XOR DST, SRC	SRC 和 DST进行逻辑异或，结果存储 DST
NOT DST	把DST替换成二进制反码

二 —— 十进制数指令

指令	说明
DAA	十进制调整
DAS	为减法进行十进制调整
AAA	为加法进行ASCII调整
AAS	为减法进行ASCII调整
AAM	为乘法进行ASCII调整
AAD	为除法进行ASCII调整

移位/循环移位指令

指令	说明
SAL/SAR DST, #	DST左移或者右移#位
SHL/SHR DST, #	DST逻辑左移或者右移#位
ROL/ROR DST, #	DST循环左移或者右移#位
RCL/RCR DST, #	通过进位位对DST移位#位

测试/比较和转移指令

指令	说明
TST SRC1, SRC2	逻辑与，根据结果设置标志位
CMP SRC1, SRC2	依SRC1-SRC2的结果设置标志位
JMP ADDR	跳转到ADDR
Jxx ADDR	基于标志执行条件转移
CALL ADDR	调用ADDR处的过程
RET	从过程返回
IRET	从中断返回
LOOP xx	循环直到条件满足
INT ADDR	初始化一个软件中断
INT0	如果溢出位被设置则发生中断

字符串指令

指令	说明
LODS	读取一个串
STOS	保存串
MOVS	复制串
CMPS	比较两个串
SCAS	扫描一个串

条件码指令

指令	说明
STC	设置EFLAGS寄存器中的进位位
CLC	清除EFLAGS寄存器中的进位位
CMC	EFLAGS中的补码进位位
STD	设置EFLAGS寄存器中的方向位
CLD	清除EFLAGS寄存器中的方向位
STI	设置EFLAGS寄存器中的中断位
CLI	清除EFLAGS寄存器中的中断位
PUSHFD	EFLAGS寄存器入栈
POPFD	EFLAGS寄存器出栈
LAHF	从EFLAGS寄存器中读取AH
SAHF	把AH保存到EFLAGS寄存器中

其它指令

指令	说明
SWAP DST	改变DST的字节顺序
CWQ	为了进行除法，把EAX扩展成EDX: EAX
CWDE	把AX中的16位数扩展成EAX
ENTER SIZE, LV	创建SIZE个字节的堆栈段
LEAVE	撤销ENTER创建的堆栈段
NOP	空操作
HLT	停机指令
IN AL, PORT	从PORT端口向AL输入一个字节
OUT PORT, AL	从AL向PORT端口输出一个字节
WAIT	等待中断